Tic Tac Toe in Java

Fall Break Programming Assignment

Breakdown – I/O, Data Structures

- Your code will allow 2 people to play tic tac toe
- The first input will be the size of the board (3 is the minimum, no maximum)
- The game will end when one player wins, or when the board is full (tie)
- Use a 2D array of char as a data structure

Rules – To improve your skills

- Do not search the web/ask others for answers
- Do not use or look for tutorials on how to write a tic tac toe in Java, or any other programming language
- Use our Java textbooks (Illuminated, Big Java) and the code that you wrote this school year
- You could also look at the Core CS textbook (Computational Thinking/problem solving algorithms)

Breakdown – Initialisation

- Input the size of the board (let's say, ***n***)
- Validate the input with a limit check
- If the input is less than 3, set the size to 3
- Create a 2D array of char, size ***[n][n]***
- Initialise the array with a value different from **O** and **X** to show that the space/cell is empty/unoccupied (minus character '-', space ' ', etc.)

Breakdown – Is the board full?

- Before you let a player make a move,
- Check if the board is full or not;
- If it is full, end the game (tie)

Breakdown – O Plays

- Input the ***row*** and ***column*** for player **O**
- Validate ***row*** and ***column*** with a range check
- Check the board's [***row***][***column***] to make sure it's empty
- If the cell is empty, place 'O' in it

Breakdown – Did O win a row?

[0][0]	[0][1]	[0][2]
[1][0]	[1][1]	[1][2]
[2][0]	[2][1]	[2][2]

Breakdown – Did O win a row?

- For each ***row*** of the board,
- Check if each ***column*** **in that row** is different from 'O'
- If it is different, O did not win the row
- If you get to the end of the row and there were not columns different from 'O', O won the row! (and game over!)

Breakdown – Did O win a column?

[0][0]	[0][1]	[0][2]
[1][0]	[1][1]	[1][2]
[2][0]	[2][1]	[2][2]

Breakdown – Did O win a column?

- For each ***column*** of the board,
- Check if that ***column*** **in each row** is different from 'O'
- If it is different, O did not win the column
- If you get to the end of the last row and there were not columns different from 'O', O won the row! (and game over!)

Breakdown – Did O win the RL diagonal?

[0][0]	[0][1]	[0][2]
[1][0]	[1][1]	[1][2]
[2][0]	[2][1]	[2][2]

Breakdown – Look for a pattern in the coordinates

- Can you see a pattern in the [row][column] coordinates of the right to left diagonal?
- Can you express this as a variable or formulae inside a loop?
- Look closely at the board and the coordinates, you should be able to find it!

Breakdown – Did O win the RL diagonal?

- For each ***row*** of the board,
- Check if the ***column*** **in the right to left diagonal** is different from 'O'
- If it is different, O did not win the right to left diagonal
- If you get to the end of the last row and last column of the RL diagonal and there were no cells different from 'O', O won the RL diagonal! (and game over!)

Breakdown – Did O win the LR diagonal?

[0][0]	[0][1]	[0][2]
[1][0]	[1][1]	[1][2]
[2][0]	[2][1]	[2][2]

Breakdown – Look for a pattern in the coordinates

- Again, look for a pattern in the [row][column] coordinates of the left to right diagonal
- Can you express this as a formulae inside a loop?
- Look closely at the board and the coordinates, you should be able to come up with it!
- This is probably the toughest challenge in this assignment!

Breakdown – Did O win the LR diagonal?

- For each ***row*** of the board,
- Check if the ***column*** **in the left to right diagonal** is different from 'O'
- If it is different, O did not win the right to left diagonal
- If you get to the end of the last row and last column of the LR diagonal and there were no cells different from 'O', O won the LR diagonal! (and game over!)

Breakdown – Now do the same for X

- After testing and making sure that your code works for O, you could copy and paste it and modify it for X
  - Note that at this time, you may not be able to test for a tie (full board with no winner) yet
- Extra challenge:
  - Is there any other way to avoid such redundancy?

Breakdown – Techniques/Code to Use

- 2D arrays ← data structure
- Methods
- *char*, *boolean* and *int* data types
- *while* loops (validation, keep playing)
- Simple *if* statements
- *for* loops; simple and nested (x2)
- Linear/sequential search (rows and columns), adapted for *tic tac toe* purposes

Take your time and work throughout this assignment yourselves!

Have a nice and useful break
Balance Rest and Work